

Universidad del valle de Guatemala

Programación orientada a objetos

Angel Javier Tum González - 26385

Análisis

Responda las siguientes preguntas:

1. ¿Qué clases son necesarias para representar el problema planteado?

OrdenServicio

Visibilidad	Atributo	Tipo	Propósito
Private	numeroOrden	int	Identifica de manera única la orden.
Private	nombrePropietario	String	Almacena el nombre del propietario.
Private	placaVehiculo	String	Almacena la placa del vehículo.
Private	descripcionServicio	String	Describe el mantenimiento.
Private	costoEstimado	double	Guarda el costo aproximado del servicio.

Métodos

Método	Retorno	Parámetros	Propósito
OrdenServicio(numeroOrden int, nombrePropietario String, placaVehiculo String, descripcionServicio String, costoEstimado double)	Constructor	numeroOrden int, nombrePropietario String, placaVehiculo String, descripcionServicio String, costoEstimado double	Inicializa una orden con los valores recibidos.
getNumeroOrden()	int	Ninguno	Devuelve el número de orden.
getNombrePropietario()	String	Ninguno	Devuelve el nombre del propietario.
getPlacaVehiculo()	String	Ninguno	Devuelve la placa.
getDescripcionServicio()	String	Ninguno	Devuelve la descripción del servicio.

getCostoEstimado()	double	Ninguno	Devuelve el costo estimado.
setDescripcionServicio(String nuevaDescripcion)	Void	String nuevaDescripcion	Cambia la descripción del servicio.
setCostoEstimado(double nuevoCosto)	void	Double nuevoCosto	Cambia la descripción del servicio.
toString()	String	Ninguno	Devuelve todos los datos de la orden como texto.

GestorOrdenes

Visibilidad	Atributo	Tipo	Propósito
Private	ordenes	List<OrdenServicio>	Almacena dinámicamente todas las órdenes activas.

Métodos

Método	Retorno	Parámetros	Propósito
GestorOrdenes()	Constructor	Ninguno	Crea la colección como un ArrayList vacío.
agregarOrdenes(OrdenServicio orden)	void	OrdenServicio orden	Agrega una orden si su número no está repetido.
obtenerOrdenes()	List<OrdenServicio>	Ninguno	Devuelve las órdenes registradas.
buscarOrden(numeroOrden int)	OrdenServicio	int numeroOrden	Busca y devuelve una orden por su número.
modificarOrden(numeroOrden int, descripcion String, costo double)	void	Int numeroOrden, String descripcion, double costo	Modifica descripción y costo.
cancelarOrden(numeroOrden int)	void	Int numeroOrden	Elimina una orden de la colección.

buscarPorPlaca(placa String)	List<OrdenServicio>	String placa	Devuelve todas las órdenes de una placa.
calcularCostoTotal()	double	Ninguno	Suma los costos de todas las órdenes.
calcularCostoPromedio()	double	Ninguno	Calcula el promedio de los costos.
obtenerOrdenMayorCosto()	OrdenServicio	Ninguno	Devuelve la orden con el costo más alto.
obtenerCantidadOrdenes()	int	Ninungo	Devuelve el tamaño de la colección.

VistaTaller

Visibilidad	Atributos	Tipo	Propósito
Private	leer	Scanner	Scanner para poder obtener la información del usuario.

Métodos

Método	Retorno	Parámetros	Propósito
mostrarMenu()	void	Ninguno	Muestra el menú.
solicitarOpcion()	int	Ninguno	Solicita un número para la opcion que el usuario desee
solicitarNumeroOrden()	int	Ninguno	Solicita el numero de la orden.
SolicitarNombrePropietario()	String	Ninguno	Solicita el nombre del propietario.
solicitarPlaca()	String	Ninguno	Solicita la placa.
solicitarDescripcion()	String	Ninguno	Solicita una descripcion.
solicitarCosto()	double	Ninguno	Solicita el costo.
mostrarOrden()	void	OrdenServicio orden	Muestra el orden.
mostrarOrdenes()	void	List<OrdenServicio> ordenes	Muestra la lista de ordenes
mostrarReporteCostos()	void	Double total, double promedio	Lista los reportes de los costos.

mostrarCantidad()	void	Int cantidad	Mostrar las cantidades
mostrarMensaje()	void	String mensaje	Mostrar los mensajes.
mostrarError()	void	String mensaje	Muestra el mensaje del error

ControladorTaller

Visibilidad	Atributos	Tipo	Propósito
Private	gestor	GestorOrdenes	Conoce con gestor ordenes
Private	vista	VistaTaller	Hace las solicitudes al usuario.

Métodos

Método	Retorno	Parámetros	Propósito
ControladorTaller()	Controller	GestorOrdenes gestor, VistsTaller vista	Controlador
registrarOrden()	void	-----	Registra la orden.
consultarOrdenes()	void	-----	Consulta las ordenes que estan.
buscarOrden()	void	-----	Busca la orden.
modificarOrden()	void	-----	Modifica el orden.
cancelarOrden()	void	-----	Cancela la orden seleccionada.
consultarPorPlaca()	void	-----	Consulta las placas.
mostrarReporteCosto()	void	-----	Muestra los reportes de costos.
mostrarOrdenMayorCosto()	void	-----	Muestra el orden mayor de los costos.
mostrarCantidadOrdenes()	void	-----	Muestra la cantidad de ordenes.

Main

Contiene la main
Main(String[] args) : void

2. ¿Qué propiedades y métodos tendrá cada clase?
3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
4. ¿Qué información deberá almacenarse dentro de la colección dinámica?

La colección dinámica almacenará objetos de la clase OrdenServicio. Cada objeto representará una orden registrada y contendrá la siguiente información:

- Número de orden de tipo int.
- Nombre del propietario de tipo String.
- Placa del vehículo de tipo String.
- Descripción del servicio de tipo String.
- Costo estimado de tipo double.

La colección estará ubicada en la clase GestorOrdenes, que forma parte del Modelo en la arquitectura MVC. No se almacenarán los atributos por separado; cada elemento de la colección será un objeto completo de tipo OrdenServicio.

5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?

En la clase GestorOrdenes, perteneciente al Modelo, se declarará la colección con `List<OrdenServicio>` y se inicializará en el constructor con `new ArrayList<>()`. También se utilizará un `ArrayList` para guardar temporalmente las órdenes encontradas por placa.

6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?

Los atributos de todas las clases serán `private` para mantener el encapsulamiento. Los constructores, getters, setters y métodos que deban ser utilizados por otras clases serán `public`. En el Controlador, los métodos auxiliares de cada opción del menú serán `private`, mientras que `iniciar()` será `public`. El método `main()` será `public static`.

7. ¿Qué parámetros serán requeridos por los métodos de sus clases?

El constructor de `OrdenServicio` recibirá el número, propietario, placa, descripción y costo. Los métodos de búsqueda y cancelación recibirán el número de orden; el de modificación recibirá el número, la nueva descripción y el nuevo costo; y la búsqueda por placa recibirá la placa. El constructor del Controlador recibirá un objeto `GestorOrdenes` y un objeto `VistaTaller`.

8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?

Los objetos `OrdenServicio` recibirán sus valores iniciales mediante su constructor, utilizando los datos ingresados por el usuario. La colección se inicializará vacía con `new ArrayList<>()`. En `Main` se crearán los objetos del Modelo, Vista y Controlador para conectarlos e iniciar el programa.

9. ¿Cómo identificará una orden específica dentro de la colección?

Cada orden se identificará mediante su número de orden, el cual deberá ser único. Para encontrarla, se recorrerá la colección y se comparará el número buscado con el número de cada objeto.

10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?

Las órdenes se agregarán mediante `ordenes.add(orden)`, después de comprobar que el número no esté repetido. Para cancelar una orden, primero se buscará por su número y luego se eliminará con `ordenes.remove(orden)`. Si no existe, se lanzará una excepción.

11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?

Para buscar por placa, se recorrerá toda la colección y se comparará la placa de cada orden con la placa ingresada mediante `equalsIgnoreCase()`. Las coincidencias se guardarán en un nuevo `ArrayList<OrdenServicio>`, porque una misma placa puede tener varias órdenes registradas.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?

La colección se recorrerá mediante un ciclo `for-each`, sumando el costo estimado de cada orden. El costo promedio se calculará dividiendo el valor total entre la cantidad de órdenes registradas. Si la colección está vacía, el promedio será cero para evitar una división inválida.

13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?

Para determinar la orden con el costo más alto, se recorrerá la colección y se comparará el costo de cada orden con el mayor costo encontrado hasta ese momento. Al finalizar, se devolverá el objeto `OrdenServicio` que tenga el costo estimado más alto. Si no existen órdenes, se informará al usuario.

14. ¿Qué situaciones dentro del programa pueden producir una excepción?

Pueden producirse excepciones al ingresar letras cuando se espera un número, registrar un costo menor o igual a cero, dejar campos vacíos, utilizar un número de orden repetido, seleccionar una opción inválida o intentar buscar, modificar o cancelar una orden inexistente.

15. ¿Qué operaciones serán protegidas utilizando `try-catch`?

Se protegerán con `try-catch` la lectura de valores numéricos y las operaciones de registro, búsqueda, modificación y cancelación. El Controlador capturará excepciones como `NumberFormatException`, `IllegalArgumentException` y `OrdenNoEncontradaException`, mientras que la Vista mostrará el mensaje correspondiente.

16. ¿Qué acción realizará mediante el bloque `finally` y por qué debe ejecutarse independientemente del resultado de la operación?

El bloque `finally` mostrará un mensaje indicando que la operación terminó y que el usuario regresará al menú principal. Debe ejecutarse independientemente del resultado porque

permite cerrar correctamente el proceso, tanto si la operación fue exitosa como si ocurrió una excepción.